

Módulo 1 — Entidades (Sin Value Objects)

Curso: Arquitectura de Software Profesional con Clean Architecture, DDD y SOLID

Caso de uso guía: **CU-CLI-001 — Alta de Cliente**

Objetivo del módulo

Entender las **entidades** como el núcleo del dominio. Aprender a construirlas correctamente: con identidad propia, comportamiento, encapsulamiento y reglas de negocio internas. Al finalizar este módulo, el alumno habrá creado la entidad

Customer con validaciones básicas, sin utilizar Value Objects todavía.

Módulo 0: modelo mental → Módulo 1: primera entidad → Módulo 2: Value Objects

Temas del módulo

Tema 1	¿Qué es una Entidad?
Tema 2	Identidad vs atributos
Tema 3	Encapsulamiento y niveles de acceso
Tema 4	Constructor privado y clase base
Tema 5	Método Create()
Tema 6	Reglas de negocio dentro de la entidad
Tema 7	Entidades anémicas vs entidades ricas

Ubicación en el proyecto

```
CommerX.sln
+---src
  +---CommerX.Domain
    +---Common
      |   +---Entities
      |       └─ BaseEntity.cs   ← clase base abstracta (se crea en este módulo)
    +---Customers
```



+++Entities

└ Customer.cs

← entidad principal (se crea en este módulo)

Tema 1 — ¿Qué es una Entidad?

1. ¿Qué es?

En Domain-Driven Design, una **Entidad** es un objeto del dominio que posee una **identidad única** que lo distingue de todos los demás objetos, incluso si sus atributos son idénticos.

La identidad de una entidad *no cambia* a lo largo del tiempo. Un cliente puede cambiar su teléfono, su dirección e incluso su nombre, pero sigue siendo el mismo cliente porque su **Id** permanece invariable.

2. Analogía

Pensemos en dos personas llamadas *Juan García*, nacidas el mismo día y en la misma ciudad. Sus atributos son idénticos, pero son personas distintas. ¿Qué las diferencia? Su **número de DNI**: una identidad única que no cambia aunque Juan cambie de nombre o de domicilio.

En software, el rol del DNI lo cumple el **Id** de la entidad. Generalmente es un **Guid** generado automáticamente al momento de creación.

3. Características esenciales

- **Identidad única:** tiene un **Id** que la distingue de todas las demás.
- **Ciclo de vida:** nace, puede modificarse y puede eliminarse.
- **Comportamiento:** no es solo un contenedor de datos; tiene métodos que representan acciones del negocio.
- **Encapsulamiento:** sus datos internos no son modificables libremente desde afuera.
- **Reglas de negocio propias:** valida sus propios datos antes de aceptarlos.

4. ¿Por qué se usa?

Las entidades son el núcleo del modelo de dominio. Colocar las reglas dentro de la entidad garantiza que:

- La regla se aplica *siempre*, sin importar desde dónde se llame.
 - No es posible crear un objeto en estado inválido.
 - El código es fácil de leer: la entidad documenta las reglas del negocio.
-

5. ¿Cómo encaja en la arquitectura?

Clean Architecture	Las entidades viven en la capa Domain . Es la capa más interna: no conoce EF Core, la API ni ninguna tecnología externa.
DDD	La entidad modela un concepto del negocio real. Customer no es una tabla: es la representación del cliente tal como el negocio lo entiende.
SOLID — SRP	La entidad tiene una única responsabilidad: representar al cliente del dominio y garantizar que sus datos sean siempre válidos.

Tema 2 — Identidad vs atributos

1. ¿Qué es?

Identidad	El Id. Define <i>quién</i> es el objeto. Es inmutable: se asigna al nacer y nunca cambia. En C# usamos <code>Guid</code> para garantizar unicidad global.
Atributos	Los demás datos: nombre, apellido, email, etc. Pueden cambiar a lo largo del ciclo de vida, pero el objeto sigue siendo el mismo porque su Id no varía.
Auditoría	Datos técnicos que registran <i>quién</i> y <i>cuándo</i> creó o modificó la entidad. Gestionados por la infraestructura, no por el negocio. Se detallan en el Módulo 8 .

2. Analogía

Una persona puede cambiarse el nombre, mudarse de ciudad y cambiar de apellido. Todos sus atributos cambiaron, pero su número de documento sigue siendo el mismo. La identidad trasciende a los atributos.

3. ¿Por qué usar Guid como Id?

Guid (Globally Unique Identifier):

- Identificador de 128 bits generado aleatoriamente.
- La probabilidad de colisión es prácticamente nula.
- No depende de la base de datos para generarse: el dominio lo genera.
- Ejemplo: 3f2504e0-4f89-11d3-9a0c-0305e82c3301

- ✓ `Guid.NewGuid()` → el dominio controla su propia identidad
- ✗ `int autoincrement` → la identidad la asigna la base de datos (dependencia externa)

Tema 3 — Encapsulamiento y niveles de acceso

1. ¿Qué es el encapsulamiento?

El **encapsulamiento** establece que los datos internos de un objeto deben estar *protegidos del acceso externo no controlado*. Toda modificación debe pasar por los métodos del propio objeto.

2. Analogía

Una caja fuerte tiene una combinación. No se puede meter ni sacar dinero sin conocerla. El encapsulamiento funciona igual: los datos están "adentro" de la clase y solo se accede a ellos a través de los métodos que la clase expone.

3. Niveles de acceso en setters: `private` vs `protected`

```

1 // — private set —————
2 // Solo la clase que declara la propiedad puede asignarle un valor.
3 // Ni siquiera las subclases pueden modificarla.
4 public string FullName { get; private set; } // solo Customer la asigna
5
6 // — protected set —————
7 // La clase que la declara Y sus subclases pueden asignarle un valor.
8 // Desde afuera de la jerarquía sigue siendo de solo lectura.
9 public Guid Id { get; protected set; } // BaseEntity y sus subclases lo asignan
  
```

¿Quién puede leer?	Cualquiera. Ambos tienen <code>public get</code> : se pueden leer desde cualquier capa.
¿Quién escribe con <code>private set</code> ?	Solo la clase donde está declarada. Usamos esto en <code>Customer</code> para sus propiedades propias: <code>FullName</code> , <code>Email</code> , etc.
¿Quién escribe con <code>protected set</code> ?	La clase base y todas sus subclases. Usamos esto en <code>BaseEntity</code> porque su constructor necesita asignar <code>Id</code> , <code>CreatedOn</code> , etc. al construir cualquier entidad hija.

🔍 ¿Por qué esta distinción importa en nuestro diseño?

- `BaseEntity` declara `Id` con `protected set` para que su propio constructor lo asigne automáticamente.
- `Customer` declara `FullName` con `private set` porque ninguna clase externa, ni una subclase, debe modificarlo directamente.



- Desde afuera (*Application, Presentation*), ambas propiedades son de **solo lectura**. El encapsulamiento se respeta en los dos casos.

Tema 4 — Constructor privado y clase base

1. ¿Qué es un constructor privado?

Un **constructor privado** marcado con `private` impide que el objeto sea instanciado directamente desde afuera con `new`.

2. Analogía

Es como una fábrica de automóviles. No se puede construir un auto en la vereda. Toda la construcción ocurre dentro de la fábrica, bajo condiciones controladas. El constructor privado es la "puerta cerrada" de esa fábrica.

3. ¿Por qué se usa en DDD?

En DDD, una entidad debe nacer **siempre en un estado válido**. Si el constructor fuera público, cualquier parte del sistema podría crear un `Customer` vacío o con datos inválidos:

```
1 // ✗ MAL: constructor público – nace sin datos, en estado inválido
2 var customer = new Customer();           // objeto vacío, sin nombre, sin email
3 customer.FullName = "Juan";             // se asigna después, sin validación
4 // si alguien olvidó asignar el email, el objeto quedó incompleto
5
6 // ✓ BIEN: constructor privado – la única forma de crear un Customer es Create()
7 var customer = Customer.Create("Juan", "García", ...); // nace completo y válido
```

4. El patrón de constructor con parámetros opcionales

En nuestro diseño, `BaseEntity` usa un constructor `protected` con **parámetros opcionales** (valor por defecto `default`). El mismo constructor cumple dos roles:

El constructor `protected` con defaults cumple dos roles:

Rol 1 – Creación de entidad nueva (dominio):

`Customer.Create(...)` ejecuta: `new Customer()` → llama a `base()`
→ todos los params usan default → `BaseEntity` genera los valores:
`Id` = `Guid.NewGuid()` (nuevo identificador único)
`CreatedOn` = `DateTime.UtcNow` (fecha y hora actuales)

Rol 2 – Materialización desde base de datos (EF Core – Módulo 8):

EF Core llama al constructor pasando los valores reales de la BD:



Id = el Guid guardado en la base de datos
CreatedOn = la fecha guardada en la base de datos

```
1 // Constructor de BaseEntity con parámetros opcionales
2 protected BaseEntity(
3     Guid id = default, // default de Guid = 00000000-0000-0000-0000-000000000000
4     Guid createdBy = default, // default de Guid = 00000000-0000-0000-0000-000000000000
5     DateTime createdOn = default) // default de DateTime = 01/01/0001 00:00:00
6 {
7     // si llegó default → generar valor nuevo; si llegó un valor real → conservarlo
8     Id = id == default ? Guid.NewGuid() : id;
9     CreatedBy = createdBy == default ? Guid.Empty : createdBy;
10    CreatedOn = createdOn == default ? DateTime.UtcNow : createdOn;
11
12    // el operador ternario de la línea 8 es equivalente a escribir:
13    // if (id == default) { Id = Guid.NewGuid(); }
14    // else { Id = id; }
15 }
```

Tema 5 — Método Create()

1. ¿Qué es?

El **método Create()** es un método estático de fábrica (*Static Factory Method*) que actúa como el **único punto de entrada** para crear una instancia válida de la entidad. Recibe todos los datos necesarios, los valida y retorna el objeto en estado correcto. Si algún dato es inválido, **lanza una excepción antes de crear el objeto**.

2. Analogía

Es como la ventanilla de admisión de un hospital. Antes de que el paciente ingrese al sistema, se verifica que traiga todos los documentos requeridos y que los datos estén completos. Si falta algo, el proceso se detiene ahí mismo.

3. Responsabilidades del método Create()

Create() tiene tres responsabilidades bien definidas:

1. Recibir los datos del exterior como parámetros.
2. Validarlos → si hay error: lanzar excepción → el objeto NO llega a crearse.
3. Construir y retornar la entidad solo si TODAS las validaciones pasaron.

4. Lo que Create() ya NO necesita hacer

Con `BaseEntity`, cuando `Create()` ejecuta `new Customer()`, el constructor privado llama a `base()` y la clase base asigna automáticamente:

<code>Id</code>	Se genera con <code>Guid.NewGuid()</code> automáticamente.
<code>CreatedOn</code>	Se asigna con <code>DateTime.UtcNow</code> automáticamente.
<code>CreatedBy</code>	Queda en <code>Guid.Empty</code> por ahora. Se actualiza en Módulo 8.
<code>ModifiedBy / ModifiedOn</code>	Quedan vacíos. Se actualizan cuando el cliente es modificado (Módulo 8).

Por lo tanto, `Create()` solo asigna las propiedades propias de `Customer`. El código queda más limpio y cada pieza cumple una única responsabilidad (SRP).

5. ¿Por qué no usar el constructor directamente con parámetros?

Nombre descriptivo	<code>Customer.Create(...)</code> comunica intención. <code>new</code> <code>Customer(...)</code> no dice qué acción del negocio representa.
Compatible con EF Core	El constructor privado queda libre para que EF Core lo use en la materialización de objetos desde la BD.
Más testeable	El comportamiento de creación queda centralizado y es fácil de probar de forma aislada.
Extensible	En el futuro puede retornar un tipo <code>Result</code> en lugar de lanzar excepción, sin cambiar la firma pública.

Tema 6 — Reglas de negocio dentro de la entidad

1. ¿Qué es?

Una **regla de negocio** es una condición que el sistema debe cumplir para reflejar correctamente el comportamiento del dominio. En DDD, esas reglas deben vivir *dentro de la entidad* que las posee, no en el controlador, no en el caso de uso y no en la base de datos.

2. Principio Fail Fast

El **principio Fail Fast** establece que ante un dato inválido, el sistema debe fallar *inmediatamente* y de forma explícita, en lugar de continuar con datos corruptos que generen errores más difíciles de rastrear.

Fail Fast: si el dato es inválido → lanzar excepción AHORA
no continuar → no guardar → no procesar datos corruptos

3. ¿Qué valida el dominio y qué no?

Nombre obligatorio	✓ Dominio. Se valida en <code>Customer.Create()</code> : lanza excepción si el nombre está vacío.
Apellido obligatorio	✓ Dominio. Se valida en <code>Customer.Create()</code> : lanza excepción si el apellido está vacío.
Mayor de 18 años	✓ Dominio. Se valida en <code>Customer.Create()</code> : lanza excepción si la edad es menor a 18.
Email único en el sistema	✗ No es dominio. Requiere consultar la base de datos. Vive en Application (Módulo 6).

💡 Límites de la responsabilidad del dominio

- El dominio valida reglas que *no requieren información externa*: datos vacíos, formato, rangos de edad.
- El dominio **no puede** validar unicidad porque necesitaría consultar la base de datos, introduciendo una dependencia de infraestructura.



- La unicidad se valida en la capa Application, *antes* de llamar a `Customer.Create()`.

Tema 7 — Entidades anémicas vs entidades ricas

1. ¿Qué es una entidad anémica?

Una **entidad anémica** es una clase que solo contiene propiedades con getters y setters públicos, sin ningún comportamiento ni validación. Es básicamente un contenedor de datos equivalente a un registro de base de datos. Martin Fowler describió este patrón como un *antipatrón*.

2. Comparación directa

```
1 // =====
2 // ❌ ENTIDAD ANÉMICA – solo datos, sin reglas
3 // =====
4 public class Customer
5 {
6     public int Id { get; set; } // ❌ setter público, cualquiera lo cambia
7     public string FullName { get; set; } // ❌ puede quedar vacío
8     public string LastName { get; set; } // ❌ puede quedar vacío
9     public string Email { get; set; } // ❌ puede tener formato inválido
10    public DateTime BirthDate { get; set; } // ❌ puede ser menor de edad
11    // sin constructor controlado, sin métodos, sin reglas
12 }
```

```
1 // =====
2 // ✅ ENTIDAD RICA – con identidad, encapsulamiento y reglas
3 // =====
4 public sealed class Customer : BaseEntity
5 {
6     public string FullName { get; private set; } = string.Empty; // ✅ protegido
7     public string Email { get; private set; } = string.Empty; // ✅ protegido
8     public DateTime BirthDate { get; private set; } // ✅ protegido
9
10    private Customer() : base() { } // ✅ constructor privado, llama a la base
11    public static Customer Create(...) { ... } // ✅ única forma de crear
12 }
```

3. Consecuencias de la entidad anémica

Reglas dispersas	Las validaciones se repiten en controladores, servicios y base de datos. Un cambio de regla requiere modificar múltiples lugares.
Estado inválido posible	<code>customer.FullName = ""</code> compila y ejecuta sin ningún error.
Violación de SRP	Los servicios y controladores acumulan lógica que debería pertenecer a la entidad.
Tests frágiles	Para testear las reglas hay que testear servicios enteros, no la entidad en aislamiento.

Ejemplo práctico — Entidad Customer (Módulo 1)

A continuación se implementan las dos clases completas de este módulo. Se utilizan tipos primitivos (`string`, `DateTime`) porque los Value Objects se introducen en el Módulo 2.

Paso 1 — Clase base BaseEntity

Archivo: CommerX.Domain / Common / Entities / BaseEntity.cs

```
1 // Clase base abstracta para todas las entidades del dominio
2 // Centraliza identidad y auditoría: todas las entidades heredan de aquí
3 namespace CommerX.Domain.Common.Entities;
4
5 public abstract class BaseEntity
6 {
7     // — Identidad —————
8     // protected set: el constructor base lo asigna; las subclases pueden leerlo
9     public Guid Id { get; protected set; }
10
11     // — Auditoría de creación —————
12     // CreatedBy: Id del usuario que creó el registro (Guid.Empty = no asignado)
13     public Guid CreatedBy { get; protected set; }
14     // CreatedOn: fecha y hora UTC en que se creó el registro
15     public DateTime CreatedOn { get; protected set; }
16
17     // — Auditoría de modificación —————
18     // ModifiedBy: Id del usuario de la última modificación (Guid.Empty = nunca modificado)
19     public Guid ModifiedBy { get; protected set; }
20     // ModifiedOn: DateTime? — el "?" indica que puede ser null (nunca modificado)
21     public DateTime? ModifiedOn { get; protected set; }
22
23     // — Constructor protegido con parámetros opcionales —————
24     // "default" como valor por defecto permite que el mismo constructor
25     // sirva para crear entidades nuevas Y para materialización con EF Core
26     protected BaseEntity(
27         Guid id = default,
28         Guid createdBy = default,
29         DateTime createdOn = default)
30     {
31         // si llegó default (todos ceros) → generar valor nuevo; si llegó real → conservarlo
32         Id = id == default ? Guid.NewGuid() : id;
33         CreatedBy = createdBy == default ? Guid.Empty : createdBy;
34         CreatedOn = createdOn == default ? DateTime.UtcNow : createdOn;
35
36         // auditoría de modificación: inicia vacía, se completa en Módulo 8
37         ModifiedBy = Guid.Empty;
38         ModifiedOn = null;
39     }
```

40 }

Puntos clave de BaseEntity:

- **abstract**: no se puede instanciar directamente — solo existe para ser heredada por **Customer**, **Order**, etc.
- **protected set**: las subclases pueden acceder a estas propiedades al construir el objeto, pero el código externo no puede modificarlas.
- **DateTime?** con signo **?**: indica que **ModifiedOn** puede ser **null**. **null** significa "nunca fue modificado".
- Los campos de auditoría usan **Guid** para representar el Id del usuario. En el Módulo 8 se verá cómo poblarlos automáticamente al guardar cambios.

Paso 2 — Entidad Customer

Archivo: CommerX.Domain / Customers / Entities / Customer.cs

```
1 // Entidad Customer – Aggregate Root del módulo Customers
2 // Módulo 1: validaciones básicas con strings (sin Value Objects todavía)
3 using CommerX.Domain.Common.Entities;
4
5 namespace CommerX.Domain.Customers.Entities;
6
7 // sealed: no puede heredarse – decisión de diseño deliberada en DDD
8 public sealed class Customer : BaseEntity
9 {
10     // — Propiedades del negocio —————
11     // private set: solo esta clase puede asignar los valores
12     // Id, CreatedBy, CreatedOn, ModifiedBy, ModifiedOn vienen de BaseEntity
13     public string FullName { get; private set; } = string.Empty;
14     public string LastName { get; private set; } = string.Empty;
15     public string Document { get; private set; } = string.Empty;
16     public string Email { get; private set; } = string.Empty;
17     public string Phone { get; private set; } = string.Empty;
18     public string Address { get; private set; } = string.Empty;
19     public DateTime BirthDate { get; private set; }
20
21     // — Constructor privado —————
22     // Impide instanciar con "new Customer()" desde afuera
23     // base() llama a BaseEntity() con todos los defaults:
24     // → Id = Guid.NewGuid(), CreatedOn = DateTime.UtcNow, etc.
25     private Customer() : base() { }
26
27     // — Método de fábrica —————
28     // Único punto de entrada para crear un Customer válido
```

```
29 // Valida todos los datos ANTES de construir el objeto (principio Fail Fast)
30 public static Customer Create(
31     string fullName,
32     string lastName,
33     string document,
34     string email,
35     string phone,
36     string address,
37     DateTime birthDate)
38 {
39     // — Validaciones (Fail Fast) —————
40     // regla: nombre obligatorio
41     if (string.IsNullOrWhiteSpace(fullName))
42         throw new ArgumentException("El nombre es obligatorio.", nameof(fullName));
43
44     // regla: apellido obligatorio
45     if (string.IsNullOrWhiteSpace(lastName))
46         throw new ArgumentException("El apellido es obligatorio.", nameof(lastName));
47
48     // regla: documento obligatorio
49     if (string.IsNullOrWhiteSpace(document))
50         throw new ArgumentException("El documento es obligatorio.", nameof(document));
51
52     // regla: email obligatorio (validación de formato en Módulo 2 con Value Objects)
53     if (string.IsNullOrWhiteSpace(email))
54         throw new ArgumentException("El email es obligatorio.", nameof(email));
55
56     // regla: el customer debe ser mayor de 18 años
57     // cálculo correcto: considera mes y día, no solo el año
58     var today = DateTime.Today;
59     var age = today.Year - birthDate.Year;
60     // si el cumpleaños de este año aún no llegó, se resta un año
61     if (birthDate.Date > today.AddYears(-age)) age--;
62     if (age < 18)
63         throw new ArgumentException("El customer debe ser mayor de 18 años.",
64     nameof(birthDate));
65
66     // — Construcción —————
67     // "new Customer()" llama al constructor privado → llama a base()
68     // BaseEntity asigna Id, CreatedOn, CreatedBy automáticamente
69     // Aquí solo se asignan las propiedades propias de Customer
70     return new Customer
71     {
72         FullName = fullName.Trim(), // se eliminan espacios innecesarios
73         LastName = lastName.Trim(), // se eliminan espacios innecesarios
74         Document = document.Trim(), // se eliminan espacios innecesarios
75         Email = email.Trim().ToLower(), // emails siempre en minúsculas
76         Phone = phone.Trim(), // se eliminan espacios innecesarios
77         Address = address.Trim(), // se eliminan espacios innecesarios
78         BirthDate = birthDate.Date // solo la fecha, sin componente de hora
79     };
80 }
```

Puntos clave del código:

- `sealed` : la entidad no puede heredarse. Evita comportamientos inesperados por subclases.
- `: BaseEntity` : hereda identidad y campos de auditoría de la clase base.
- `private Customer() : base() { }` : el constructor privado llama a la base, que genera `Id` y `CreatedOn` automáticamente.
- En el `return new Customer { ... }` ya no es necesario asignar `Id` : lo hizo el constructor.
- `nameof(fullName)` : el nombre del parámetro se incluye en la excepción para facilitar el diagnóstico.
- El cálculo de edad considera mes y día, evitando el error clásico de solo restar años.
- En el Módulo 2 estas validaciones primitivas serán reemplazadas por Value Objects especializados.

Actividades prácticas — Módulo 1

Las actividades de este módulo requieren escribir código C# por primera vez. Se trabaja directamente en el proyecto `CommerX.Domain`.

Actividad 1 — Crear la clase base BaseEntity

Consigna:

Crear el archivo `BaseEntity.cs` en la ruta indicada y escribir la clase base abstracta exactamente como se muestra en el ejemplo práctico.

Ruta: `CommerX.Domain / Common / Entities / BaseEntity.cs`

Preguntas de reflexión:

1. ¿Por qué el constructor de `BaseEntity` es `protected` y no `private`?
2. ¿Por qué `ModifiedOn` es `DateTime?` (nullable) mientras que `CreatedOn` es `DateTime` (no nullable)?
3. Si un `Customer` nunca fue modificado, ¿cuál es el valor de `ModifiedOn`? ¿Cómo lo verificarías en el código con un `if`?
4. ¿Qué ventaja tiene usar parámetros con valor `default` en lugar de dos constructores separados?

Actividad 2 — Crear la entidad Customer con validaciones básicas

Consigna:

Crear el archivo `Customer.cs` comenzando solo con las validaciones de nombre y apellido obligatorios. Las demás se agregan en pasos siguientes.

Ruta: `CommerX.Domain / Customers / Entities / Customer.cs`

Requisitos obligatorios:

- La clase debe heredar de `BaseEntity`.
- El constructor debe ser `private` y llamar a `base()`.
- Las propiedades propias deben tener `private set`.
- El método `Create()` debe ser `public static`.
- Si el nombre está vacío, lanzar `ArgumentException` con mensaje descriptivo.
- Si el apellido está vacío, lanzar `ArgumentException` con mensaje descriptivo.
- El inicializador del `return` no debe asignar `Id` ni `CreatedOn`.

Actividad 3 — Agregar validación de mayoría de edad

Consigna:

Agregar al método `Create()` la validación de que el customer sea mayor de 18 años, utilizando

el cálculo correcto que considera mes y día.

Preguntas de reflexión:

1. ¿Por qué no alcanza con restar `DateTime.Now.Year - birthDate.Year`? Dar un ejemplo concreto de fecha de nacimiento que produciría un resultado incorrecto.
2. Estamos usando `ArgumentException`. ¿Sería más correcto tener una excepción propia como `InvalidAgeException`? ¿Por qué? (Este tema se resuelve en el Módulo 3.)

Actividad 4 — Verificar el encapsulamiento

Consigna:

Escribir el siguiente código en un proyecto de consola de prueba y verificar que las líneas marcadas **no compilan**:

```
var c1 = new Customer();           // debe NO compilar
var c2 = Customer.Create("Juan", ...); // debe compilar
c2.FullName = "Pedro";             // debe NO compilar
c2.Id = Guid.NewGuid();            // debe NO compilar
```

Preguntas:

1. ¿Qué mensaje de error del compilador aparece al intentar `new Customer()`?
2. ¿Por qué `c2.Id = ...` tampoco compila si `Id` tiene `protected set` y no `private set`?
3. Si el negocio necesitara cambiar el nombre de un customer ya registrado, ¿qué método habría que agregar a la entidad? ¿Cómo lo llamarías y qué validaciones haría?

Actividad 5 — Reflexión: entidad anémica vs rica

Consigna:

Dado el siguiente código de entidad anémica, reescribirlo como entidad rica aplicando todos los principios del módulo:

```
public class Order
{
    public int Id { get; set; }
    public string CustomerId { get; set; }
    public decimal Total { get; set; }
    public DateTime Date { get; set; }
    public string Status { get; set; }
}
```

Requisitos de la reescritura:

- Heredar de `BaseEntity`.
- Constructor privado que llame a `base()`.
- Método `Create()` con al menos una validación de negocio.
- Todas las propiedades propias con `private set`.
- Regla obligatoria: el total no puede ser menor o igual a cero.

- El inicializador del `return` no debe asignar `Id`.

Pregunta de reflexión:

1. ¿Qué campos de auditoría hereda `Order` automáticamente de `BaseEntity`? ¿Es necesario declararlos nuevamente en `Order`?

✓ Al completar este módulo, el alumno tiene su primera entidad de dominio funcional con identidad propia, encapsulamiento real y auditoría integrada desde la clase base. En el **Módulo 2 — Value Objects**, las validaciones primitivas serán reemplazadas por objetos especializados más robustos y reutilizables.